

DS289: Assignment-1: Report

Parikshit Mahajan

February 16, 2025

Question 1

- (a) For a function $u(x)$, derive a second-order accurate finite difference approximation to compute

$$\frac{d^2u}{dx^2}$$

at a point i on a non-uniform grid. The scheme should use a one-sided stencil to the right of i .

- (b) Derive a second-order accurate finite difference approximation to compute the cross derivative

$$\frac{\partial^2 u}{\partial x \partial y}$$

of a function $u(x, y)$ on a uniform grid. Obtain the leading-order terms in the truncation error.

Solution

- (a) For a non-uniform grid with a one-sided stencil, the general finite difference formula is:

$$\frac{d^a u}{dx^a} = \sum_{j=J_1}^{J_2} C_j u_{i+j} + O(\Delta x^r). \quad (1)$$

Setting $J_1 = 0$ gives a forward difference scheme. To achieve second-order accuracy, we choose $a = 2$ and $r = 2$, yielding:

$$\frac{d^2 u}{dx^2} = c_0 u_i + c_1 u_{i+1} + c_2 u_{i+2} + c_3 u_{i+3} + O(\Delta x^2).$$

Expanding using Taylor series:

$$\begin{aligned} c_3 \left[u_{i+3} &= u_i + u'_i \sum_{j=1}^3 \Delta x_j + \frac{u''_i}{2} \left(\sum_{j=1}^3 \Delta x_j \right)^2 + \frac{u'''_i}{3!} \left(\sum_{j=1}^3 \Delta x_j \right)^3 + \dots \right] \\ c_2 \left[u_{i+2} &= u_i + u'_i \sum_{j=1}^2 \Delta x_j + \frac{u''_i}{2} \left(\sum_{j=1}^2 \Delta x_j \right)^2 + \frac{u'''_i}{3!} \left(\sum_{j=1}^2 \Delta x_j \right)^3 + \dots \right] \\ c_1 \left[u_{i+1} &= u_i + u'_i \Delta x_1 + \frac{u''_i}{2} (\Delta x_1)^2 + \frac{u'''_i}{3!} (\Delta x_1)^3 + \dots \right] \\ c_0 u_i &= u_i. \end{aligned}$$

Now we can write this in matrix form and on simplifying the terms we get as follows :

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & \Delta x_1 & \Delta x_1 + \Delta x_2 & \Delta x_1 + \Delta x_2 + \Delta x_3 \\ 0 & \Delta x_1^2 & (\Delta x_1 + \Delta x_2)^2 & (\Delta x_1 + \Delta x_2 + \Delta x_3)^2 \\ 0 & \Delta x_1^3 & (\Delta x_1 + \Delta x_2)^3 & (\Delta x_1 + \Delta x_2 + \Delta x_3)^3 \end{bmatrix}$$

Substitutions: $\Delta x_1 = p, \quad \Delta x_1 + \Delta x_2 = q, \quad \Delta x_1 + \Delta x_2 + \Delta x_3 = r$

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & p & q & r \\ 0 & p^2 & q^2 & r^2 \\ 0 & p^3 & q^3 & r^3 \end{bmatrix}$$

so the matrix form become $Ac = b$:

Substitutions: $\Delta x_1 = p, \quad \Delta x_1 + \Delta x_2 = q, \quad \Delta x_1 + \Delta x_2 + \Delta x_3 = r$

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & p & q & r \\ 0 & p^2 & q^2 & r^2 \\ 0 & p^3 & q^3 & r^3 \end{bmatrix}$$

$$c = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0 \\ 2 \\ 0 \end{bmatrix}$$

$$Ac = b$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & p & q & r \\ 0 & p^2 & q^2 & r^2 \\ 0 & p^3 & q^3 & r^3 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 2 \\ 0 \end{bmatrix}$$

By solving the above matrix using Sympy library in python for which the code is given in the python file of question 1, we get the following solutions:

$$c_0 = \frac{2(p+q+r)}{pqr}$$

$$c_1 = \frac{2(-q-r)}{p(p-q)(p-r)}$$

$$c_2 = \frac{2(p+r)}{q(p-q)(q-r)}$$

$$c_3 = \frac{2(-p-q)}{r(pq-pr-qr+r^2)}$$

Hence, Second order accurate finite difference approximation to compute $\frac{d^2 u}{dx^2}$ at a point i on a non uniform grid using right handed stencil :

$$\frac{d^2 u}{dx^2} = \frac{2(p+q+r)}{pqr}u_i + \frac{2(-q-r)}{p(p-q)(p-r)}u_{i+1} + \frac{2(p+r)}{q(p-q)(q-r)}u_{i+2} + \frac{2(-p-q)}{r(pq-pr-qr+r^2)}u_{i+3} + O(\Delta x^2)$$

- (b) We want to find the a second order accurate finite difference approximation to compute the cross derivative of function $u(x, y)$:

$$\frac{\partial^2 u}{\partial x \partial y}$$

As we can use two sided stencil on a uniform grid, so we will be using second order central difference scheme for two variables x, y . we know to compute a^{th} derivative with r^{th} order accuracy we required function evaluations at $a+r-1$ points in case of one variable. As we have two variables

x, y and $a = 2, r = 2$ hence 5 number of function evaluations considering the symmetry is required. Taylor's series expansion for two variables is given by following formula:

$$u(x, y) = u(a, b) + \sum_{n=1}^{\infty} \frac{1}{n!} \sum_{i+j=n} (x-a)^i (y-b)^j \frac{\partial^n u}{\partial x^i \partial y^j}(a, b)$$

Now, consider the grid spacing: $\Delta x = x_{i+1} - x_i = h$, $\Delta y = y_{i+1} - y_i = h$

So Taylor's series expansions for five symmetric points are as follows:

$$c_1 \left[u_{i+1,j+1} = u_{i,j} + \Delta x \frac{\partial u}{\partial x} + \Delta y \frac{\partial u}{\partial y} + \frac{(\Delta x)^2}{2!} \frac{\partial^2 u}{\partial x^2} + \frac{(\Delta y)^2}{2!} \frac{\partial^2 u}{\partial y^2} + \Delta x \Delta y \frac{\partial^2 u}{\partial x \partial y} + O(\Delta x^3) \right]$$

$$c_2 \left[u_{i-1,j-1} = u_{i,j} - \Delta x \frac{\partial u}{\partial x} - \Delta y \frac{\partial u}{\partial y} + \frac{(\Delta x)^2}{2!} \frac{\partial^2 u}{\partial x^2} + \frac{(\Delta y)^2}{2!} \frac{\partial^2 u}{\partial y^2} + \Delta x \Delta y \frac{\partial^2 u}{\partial x \partial y} + O(\Delta x^3) \right]$$

$$c_3 \left[u_{i-1,j+1} = u_{i,j} - \Delta x \frac{\partial u}{\partial x} + \Delta y \frac{\partial u}{\partial y} + \frac{(\Delta x)^2}{2!} \frac{\partial^2 u}{\partial x^2} + \frac{(\Delta y)^2}{2!} \frac{\partial^2 u}{\partial y^2} - \Delta x \Delta y \frac{\partial^2 u}{\partial x \partial y} + O(\Delta x^3) \right]$$

$$c_4 \left[u_{i+1,j-1} = u_{i,j} + \Delta x \frac{\partial u}{\partial x} - \Delta y \frac{\partial u}{\partial y} + \frac{(\Delta x)^2}{2!} \frac{\partial^2 u}{\partial x^2} + \frac{(\Delta y)^2}{2!} \frac{\partial^2 u}{\partial y^2} - \Delta x \Delta y \frac{\partial^2 u}{\partial x \partial y} + O(\Delta x^3) \right]$$

$$c_5 u_{i,j} = u_{i,j}$$

Now taking a linear combination of these points and placing them in the equation-1, we get:

$$\frac{\partial^2 u}{\partial x \partial y} = c_1 u_{i+1,j+1} + c_2 u_{i-1,j-1} + c_3 u_{i-1,j+1} + c_4 u_{i+1,j-1} + c_5 u_{i,j} + O(\Delta x^3)$$

Now, using the constraints of the central difference scheme that the coefficients of the lower order derivatives must be zero and the coefficient of $\frac{\partial^2 u}{\partial x \partial y} = 1$, we will get matrix form.

So, on equating coefficient of $u_{i,j}$ to zero, we get,

$$(c_1 + c_2 + c_3 + c_4 + c_5)h = 0$$

on equating coefficient of $\frac{\partial u}{\partial x}$ to zero, we get,

$$(c_1 - c_2 - c_3 + c_4)h = 0$$

on equating coefficient of $\frac{\partial u}{\partial y}$ to zero, we get,

$$(c_1 - c_2 + c_3 - c_4)h = 0$$

on equating coefficient of $\frac{\partial^2 u}{\partial x^2}$ to zero, we get,

$$(c_1 + c_2 + c_3 + c_4) \frac{h^2}{2} = 0$$

Note that here on equating coefficient of $\frac{\partial^2 u}{\partial y^2}$ to zero results in the same equation as above, hence nothing new information is obtained.

on equating coefficient of $\frac{\partial^2 u}{\partial x \partial y}$ to one, we get,

$$(c_1 + c_2 - c_3 - c_4) = \frac{1}{h^2} = \frac{1}{\Delta x \Delta y}$$

In matrix form it will be written as follows:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & -1 & 1 & 0 \\ 1 & -1 & 1 & -1 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & -1 & -1 & 0 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \frac{1}{\Delta x \Delta y} \end{bmatrix}$$

On solving this matrix we get:

$$c_1 = \frac{1}{4\Delta x \Delta y}, \quad c_2 = \frac{1}{4\Delta x \Delta y}, \quad c_3 = -\frac{1}{4\Delta x \Delta y}, \quad c_4 = -\frac{1}{4\Delta x \Delta y}, \quad c_5 = 0$$

Thus, the second order accurate finite difference approximation to compute the cross derivative $\frac{\partial^2 u}{\partial x \partial y}$ of a function $u(x, y)$ on a uniform grid is given by:

$$\left. \frac{\partial^2 u}{\partial x \partial y} \right|_{i,j} = \frac{1}{4\Delta x \Delta y} [u_{i+1,j+1} - u_{i-1,j-1} - u_{i-1,j+1} + u_{i+1,j-1}] + O(\Delta x^3)$$

Since it is a uniform grid, so $\Delta x = \Delta y = h$. Thus, the above equation can be written as:

$$\left. \frac{\partial^2 u}{\partial x \partial y} \right|_{i,j} = \frac{1}{4h^2} [u_{i+1,j+1} + u_{i-1,j-1} - u_{i-1,j+1} - u_{i+1,j-1}] + O(\Delta x^3)$$

Note that for Taylor series expansion of two variables, $O(\Delta x^3)$ term is also nullified as it has same set of signs as that of $O(\Delta x)$. Hence the leading truncated term in Taylor series is of $O(\Delta x^4)$. So, Truncation error leading-order term will be,

$$\frac{1}{h^2} \left[\left(\frac{h^4}{24} \right) \frac{\partial^4 u}{\partial x^4} + \left(\frac{h^4}{24} \right) \frac{\partial^4 u}{\partial y^4} + \left(\frac{h^4}{4} \right) \left(\frac{\partial^4 u}{\partial x^2 \partial y^2} \right) + \left(\frac{h^4}{6} \right) \left(\frac{\partial^4 u}{\partial x^3 \partial y} \right) + \left(\frac{h^4}{6} \right) \left(\frac{\partial^4 u}{\partial x \partial y^3} \right) \right]$$

Thus, we got the truncation error leading-order term of $O(\Delta x^2)$.

Question 2

Consider the initial value problem over the interval $t = 0$ to 2.0 where $y(0) = 1$.

$$\frac{dy}{dt} = yt^2 - 1.1y \quad (1)$$

- (a) Solve this ODE analytically and plot the solution.
- (b) Obtain the numerical solution for the step sizes $h = 0.5, 0.25, 0.125, 0.0625, 0.03125$ using:
 - i. Euler's explicit method.
 - ii. Second-order Adam's-Bashforth.
 - iii. Second-order RK method (any value of dof).
- (c) Compute the maximum and average absolute errors for each step size using the analytical solution and obtain the error graph demonstrating the order of accuracy for each method. Note: use separate graphs for maximum and average errors to compare all three methods.
- (d) Discuss the results.

Solution

- (a) The analytical solution of the given IVP can be obtained as follows:

$$\frac{dy}{dt} = (t^2 - 1.1)y$$

$$\frac{dy}{dt} = (t^2 - 1.1)$$

On integrating, given initial condition $y(0) = 1$, we get,

$$\int_1^y \frac{dy}{y} = \int_0^t (t^2 - 1.1)dt$$

$$\ln y = \frac{t^3}{3} - 1.1t$$

$$y = e^{\frac{t^3}{3} - 1.1t}$$

Thus the analytical solution is as follows:

$$y(t) = e^{\frac{t^3}{3} - 1.1t}$$

The plot for the analytical solution is as follows in figure 1:

From the graph, we can see that the solution to the IVP is first decreasing in the interval $[0, \sqrt{1.1}]$, and then increasing onwards for the interval $[\sqrt{1.1}, 2]$. Thus, minimum value of $y(t)$ is at $\sqrt{1.1}$.

- (b)

The numerical solution for the step sizes $h = 0.5, 0.25, 0.125, 0.0625$, and 0.03125 using:

i. Euler's Explicit Method

Euler's explicit method is given by:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

where h is the step size.

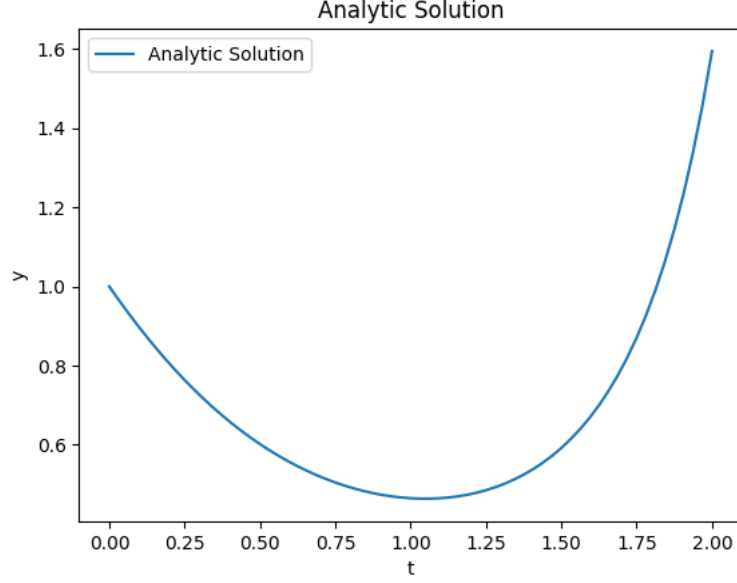


Figure 1: Analytical solution for given IVP

ii. Second-Order Adams-Bashforth Method

The second-order Adams-Bashforth method is given by:

$$y_{n+1} = y_n + \frac{h}{2} (3f(t_n, y_n) - f(t_{n-1}, y_{n-1}))$$

iii. Second-Order Runge-Kutta (RK) Method

The second-order Runge-Kutta method is given by:

$$y_{n+1} = y_n + \phi(t_n, y_n, h)h$$

$$\phi(t_n, y_n, h) = b_1 k_1 + b_2 k_2$$

So,

$$y_{n+1} = y_n + (b_1 k_1 + b_2 k_2)h$$

where,

$$k_1 = f(t_n, y_n)$$

&

$$k_2 = f(t_n + c_2 h, y_n + a_{21} k_1 h)$$

we have 4 unknowns c_2, a_{21}, b_1, b_2 and 3 equations, so we have one degree of freedom; hence we chose $c_2 = 1$, and then obtain the rest of the unknown values as follows:

$$b_1 = b_2 = \frac{1}{2}, a_{21} = 1$$

Thus, we get,

$$y_{n+1} = y_n + \frac{h}{2} (f(t_n, y_n) + f(t_n + h, y_n + hf(t_n, y_n)))$$

The corresponding to each method, code files are given in "q2" folder. Below are the plots for each method consisting of all step sizes.

- (c) The maximum and average absolute errors for each method for each step size using the analytical solution are given in the following tables. The error graphs demonstrating the order of accuracy for each method are given below. The plots for numerical solution by each method for all step sizes are as follows:

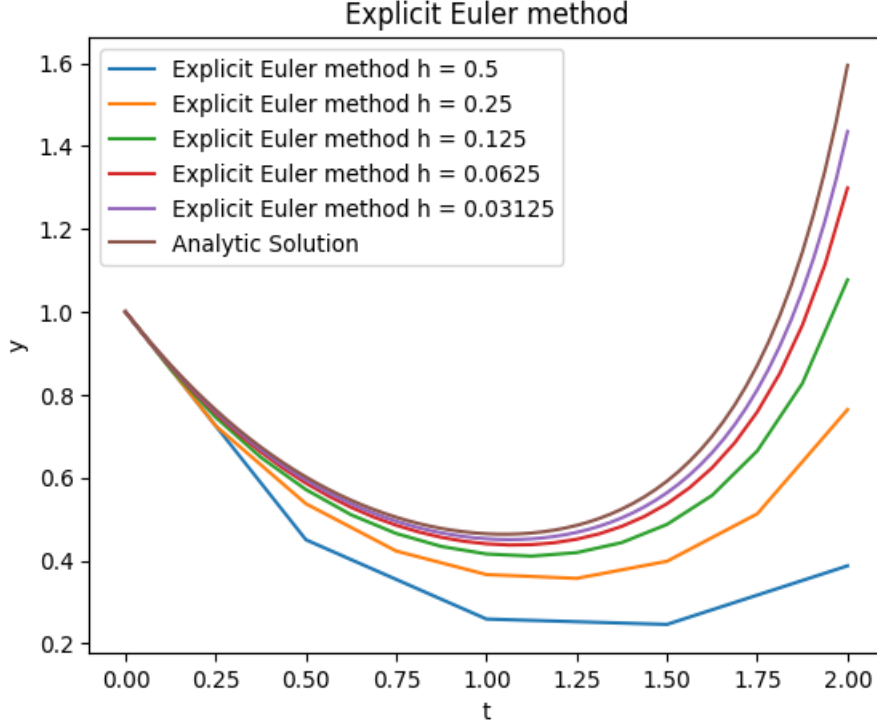


Figure 2: Explicit Euler Method : Numerical solution

- (d) Note that Explicit Euler is a one-step method, so it is self-starting. While second order Adam's Bashforth is a multi-step method; hence, it is not self-starting. It requires the one previous point function evaluation. Therefore, without compromising its order of accuracy which is $o(\Delta x^3)$ in local truncation error and one order lower $o(\Delta x^2)$ in global truncation error due to accumulation of errors. We used the RK-2 having global truncation error of order $o(\Delta x^2)$ method to approximate the first point and then use second order Adam's Bashforth method for approximating the rest of the points.

As we can see from figures 2, 7 and from table 1 that, as step size is decreasing the errors(truncation errors) are decreasing for explicit Euler method and it is approaching closer to the analytic solution, though it is a case of trade off between round-off error and truncation errors that will decide either more decrement of step size leads to closer to analytic solution or deviates from the analytic solution. from the figure 6, we can clearly inferred that order of accuracy of explicit Euler method is $o(\Delta x)$ in terms of global truncation error.

Runge-Kutta second order method and Adam's Bashforth second order method, both have same order of accuracy $o(\Delta x^2)$ in global truncation error, but AB-2 relatively depends on one extra previous point function evaluation which leads to faster accumulation of error. Hence, AB-2 method is less accurate than RK-2 method which is confirmed by the tables 2 and 3. After the certain point in interval $[0, 2]$ of t , we can see from the figure 6 that accumulation of error in AB-2 leads to deviation of its order of accuracy.i.e. slope being less than 2.

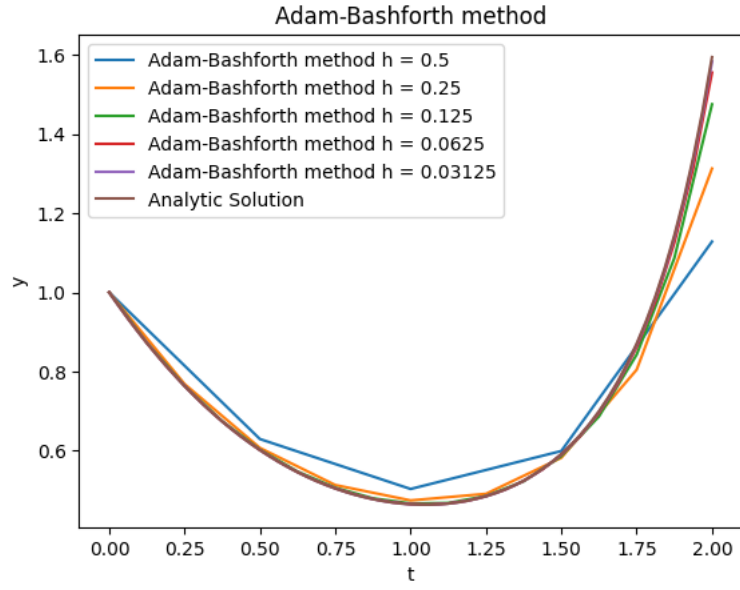


Figure 3: Second order Adam's-Bashforth Method : Numerical solution

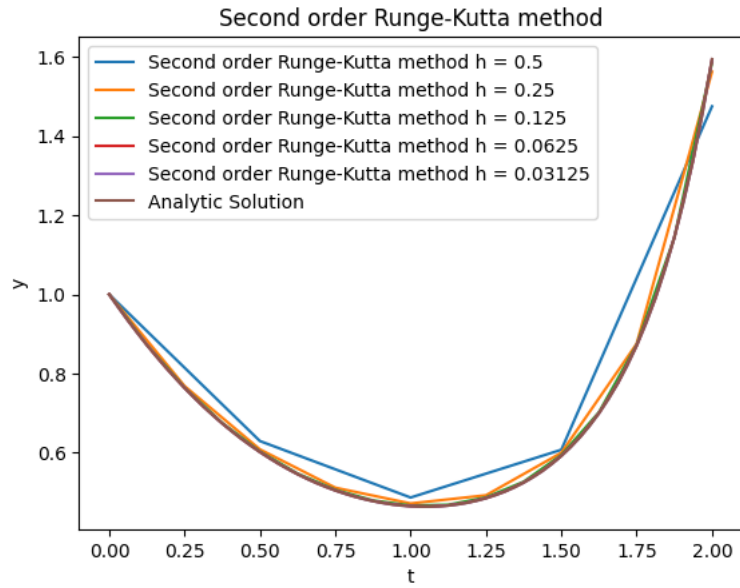


Figure 4: Second order RK Method : Numerical solution

Step Size	Max Error	Average Error
0.50000	1.20751	0.38211
0.25000	0.83056	0.19923
0.12500	0.51745	0.10232
0.06250	0.29594	0.05195
0.03125	0.15957	0.02619

Table 1: Maximum and Average Errors for Explicit Euler Method

Step Size	Max Error	Average Error
0.50000	0.46617	0.10805
0.25000	0.28159	0.04357
0.12500	0.11914	0.01402
0.06250	0.03893	0.00392
0.03125	0.01103	0.00102

Table 2: Maximum and Average Errors for second order Adam's-Bashforth Method

Step Size	Max Error	Average Error
0.50000	0.11926	0.03699
0.25000	0.03160	0.00866
0.12500	0.00850	0.00206
0.06250	0.00220	0.00051
0.03125	0.00056	0.00013

Table 3: Maximum and Average Errors for RK-2 Method

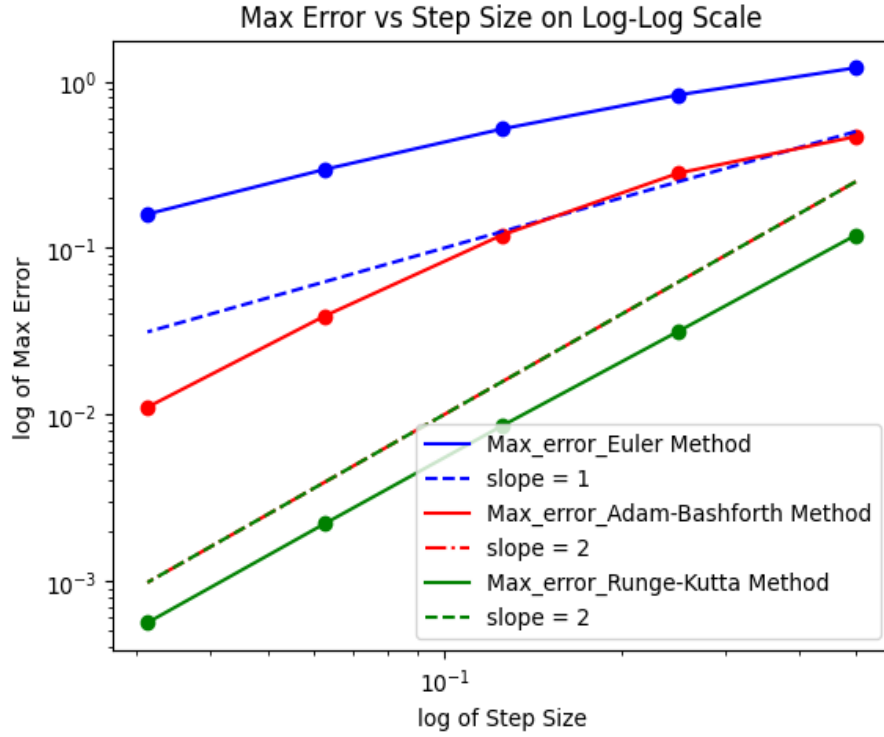


Figure 5: Maximum Error Vs Step size on log log scale

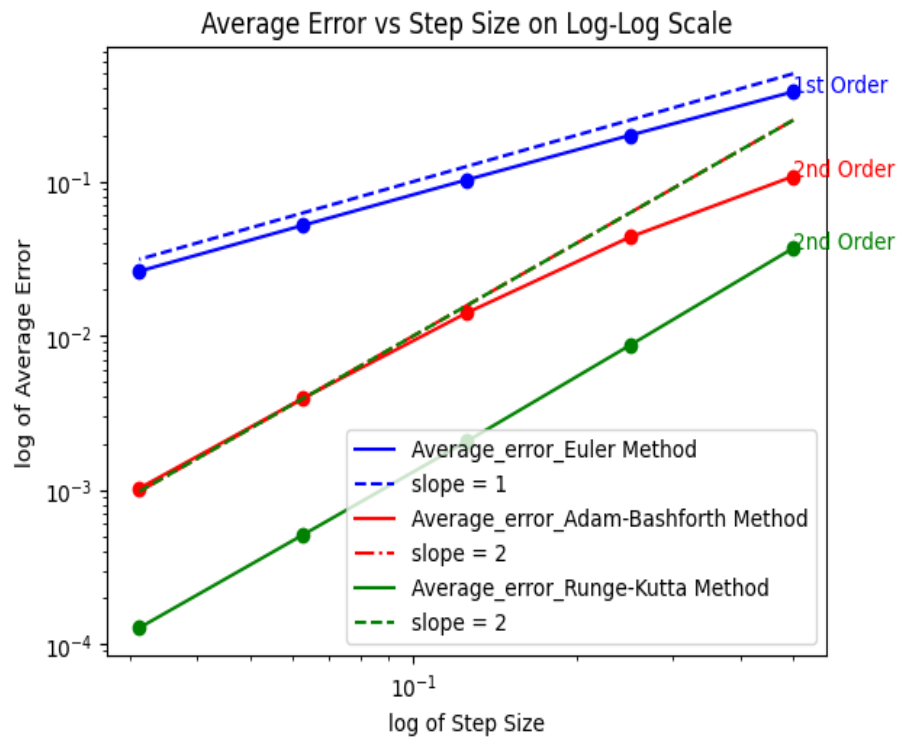


Figure 6: Average Error Vs Step size on log-log scale

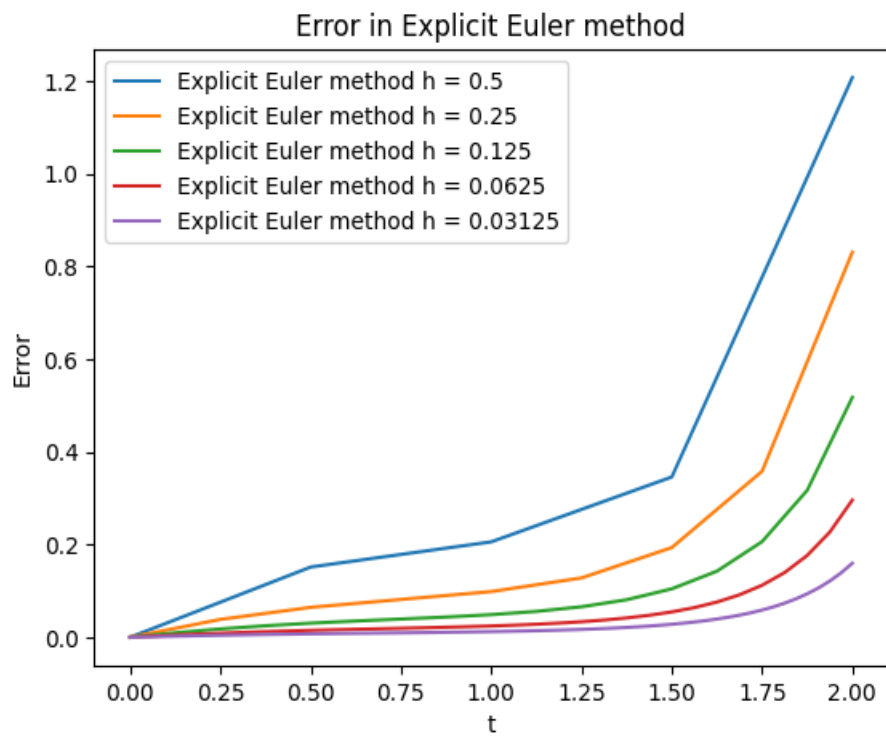


Figure 7: Error in Explicit Euler method



Figure 8: Error in Adam's-Bashforth-2 method

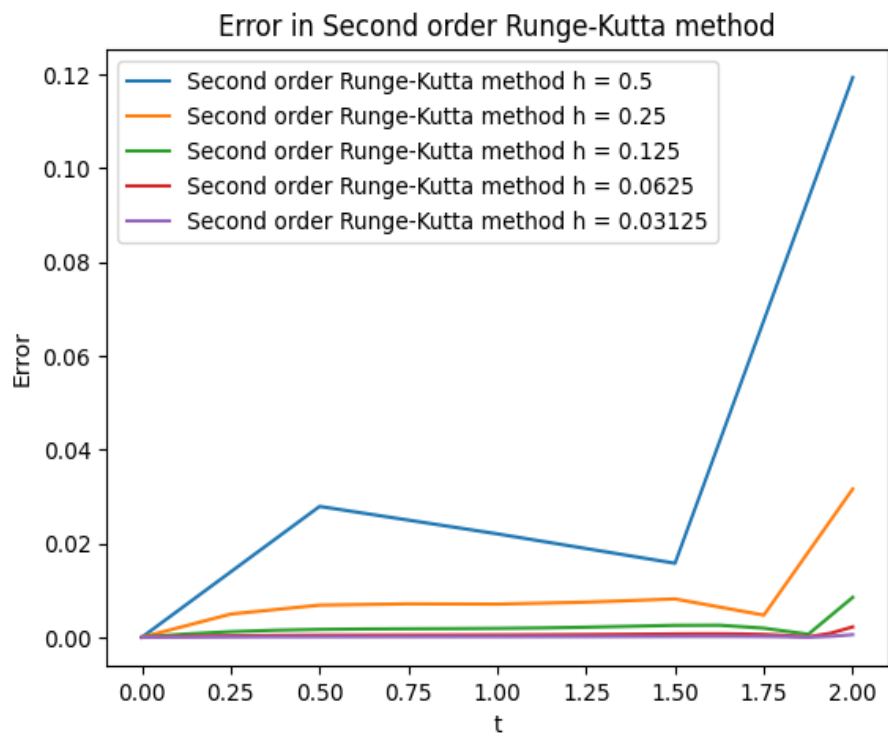


Figure 9: Error in RK-2 method

Question 3

The following ODE describes the motion of a forced damped spring-mass system:

$$m \frac{d^2x}{dt^2} + a \left| \frac{dx}{dt} \right| \frac{dx}{dt} + kx = F_0 \sin(\omega t)$$

where $x \rightarrow$ displacement from equilibrium position (m), $t \rightarrow$ time (s), $m = 2$ kg, $a = 5$ N/(m/s)², and $k = 6$ N/m. The initial velocity is zero, and the initial displacement is $x = 1$ m. The values of F_0 and ω are 2.5 N and 0.5 rad/sec, respectively.

- Solve this equation using the RK-4 method over the time period $0 \leq t \leq 15$ s.
- Starting with time step $\Delta t = 0.5$ s, progressively reduce the time step size by half to show the convergence/grid independence of results. Plot the velocity-time and displacement-time curves, and discuss your observations.
- Find the maximum and minimum values of velocity and displacement, and the corresponding times.

Solution

- (a) Given: $m = 2$ kg, $a = 5$ N/(m/s)², $k = 6$ N/m, $x(0) = 1$ m, $\frac{dx}{dt}(0) = 0$, $F_0 = 2.5$ N, $\omega = 0.5$ rad/sec.

Rewriting the given equation, we get:

$$2 \frac{d^2x}{dt^2} + 5 \left| \frac{dx}{dt} \right| \frac{dx}{dt} + 6x = 2.5 \sin(0.5t)$$

Clearly this is a second-order differential equation with two initial conditions $x(0) = 1$ and $\frac{dx}{dt}(0) = 0$.

Thus, we will first convert this second-order differential equation into system of two first-order ordinary differential equations which are as follows:

$$\frac{dx}{dt} = v$$

$$\frac{dv}{dt} = \frac{2.5 \sin(0.5t) - 5|v|v - 6x}{2}$$

with initial conditions $x(0) = 1$ and $v(0) = 0$.

The numerical solution using the RK4 method over the time period $0 \leq t \leq 15$ s for a system of two ordinary Differential Equations is obtained using the code given in the cpp file in folder "q3". The numerical solution for the step size $h = 0.03125$ using the RK-4 method is given in below figure 12. In addition to this, velocity vs time plot by RK-4 method for step size $h = 0.03125$ is given in figure 13.

As given second order ODE is non linear due to the term $a|v|v$, however python Scipy library is so powerful that it provides several powerful numerical solvers for solving some non-linear ODEs, which closely resembles to its analytical solution. Actually, this library uses *solve_ivp* function to choose best suitable numerical method among the available methods like:

1. Adaptive Runge-Kutta method
2. BDF : Backward Differentiation Formula, multi-step method ,etc.

The code for the implementation of Scipy library is given in the python notebook of folder "q3". Below are the obtained analytical solution plots for displacement vs time, refer figure 10 and for velocity vs time refer figure 11

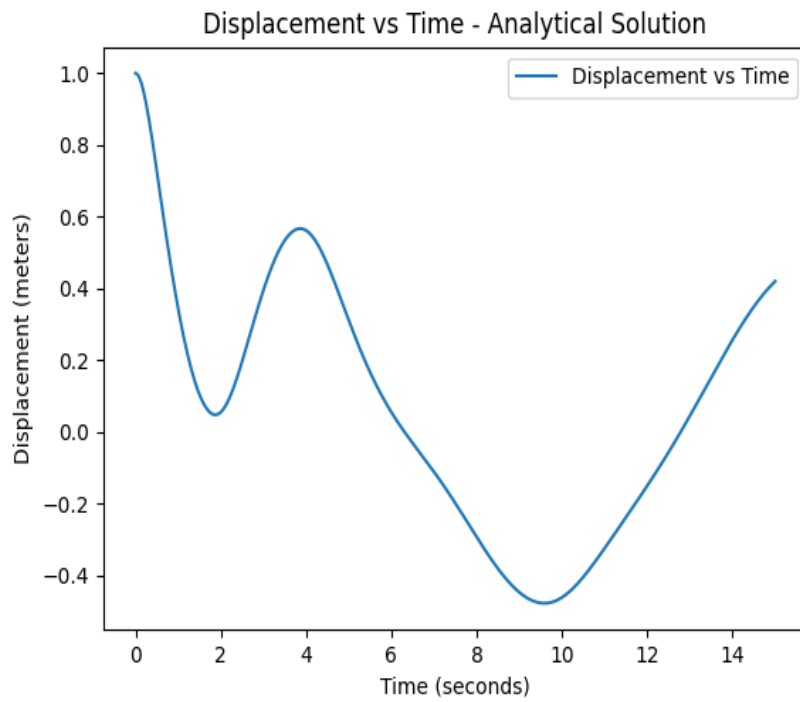


Figure 10: Analytical Solution: Displacement Vs Time

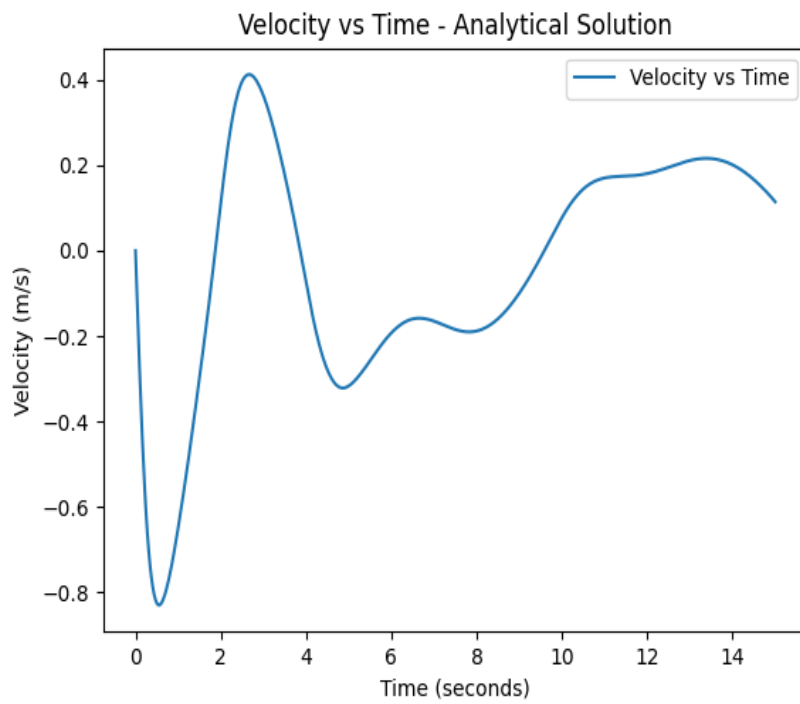


Figure 11: Analytical Solution : Velocity Vs Time

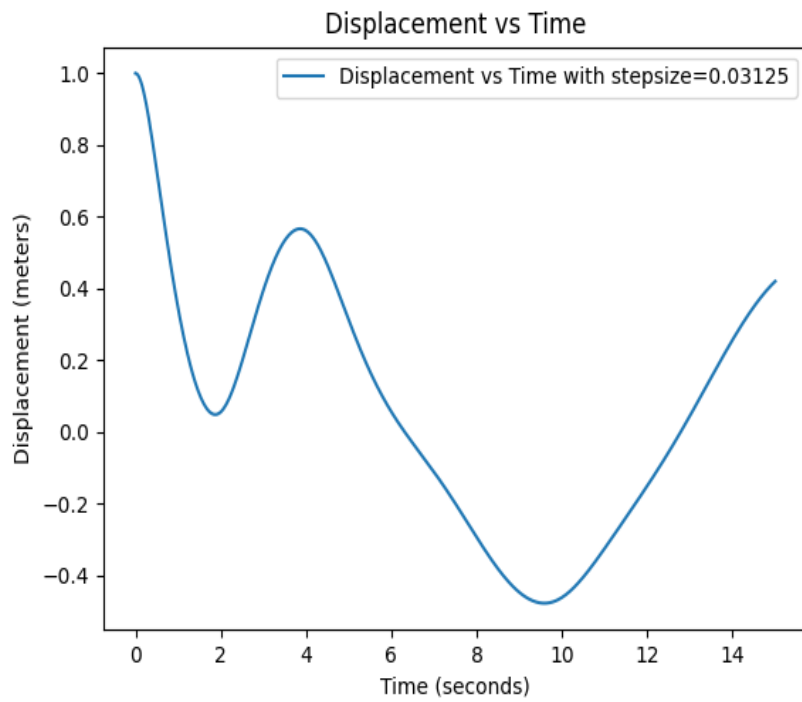


Figure 12: Displacement Vs time by RK-4 method for step size = 0.03125

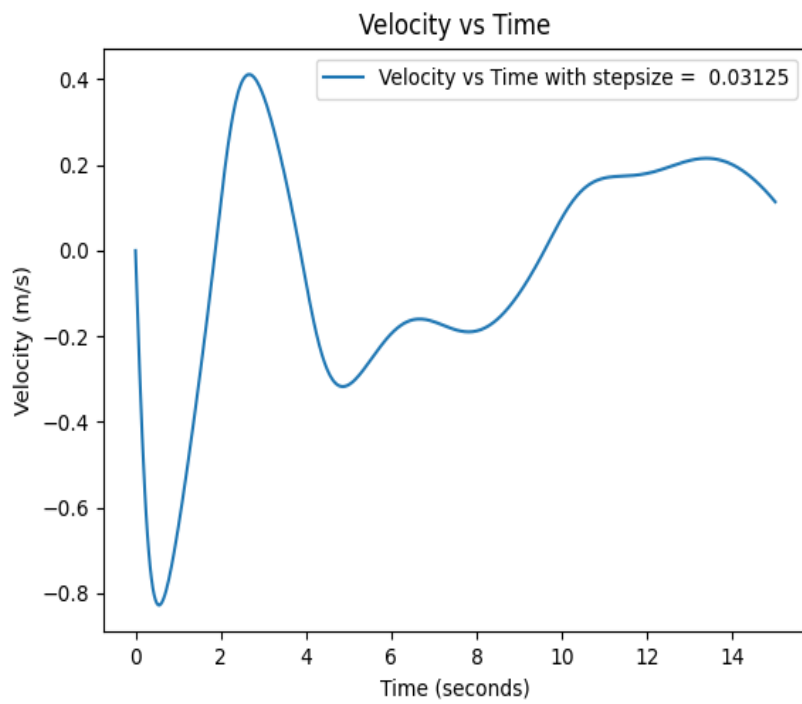


Figure 13: Velocity Vs time by RK-4 method for step size = 0.03125

The solution methodology for a System of two Ordinary Differential Equations using the RK4 method is as follows:

$$\begin{aligned}
k_{11} &= f_1(\beta, t_n, x_n, v_n) \\
k_{12} &= f_2(\beta, t_n, x_n, v_n) \\
k_{21} &= f_1\left(\beta, t_n + \frac{h}{2}, x_n + k_{11}\frac{h}{2}, v_n + k_{12}\frac{h}{2}\right) \\
k_{22} &= f_2\left(\beta, t_n + \frac{h}{2}, x_n + k_{11}\frac{h}{2}, v_n + k_{12}\frac{h}{2}\right) \\
k_{31} &= f_1\left(\beta, t_n + \frac{h}{2}, x_n + k_{21}\frac{h}{2}, v_n + k_{22}\frac{h}{2}\right) \\
k_{32} &= f_2\left(\beta, t_n + \frac{h}{2}, x_n + k_{21}\frac{h}{2}, v_n + k_{22}\frac{h}{2}\right) \\
k_{41} &= f_1(\beta, t_n + h, x_n + k_{31}h, v_n + k_{32}h) \\
k_{42} &= f_2(\beta, t_n + h, x_n + k_{31}h, v_n + k_{32}h) \\
x_{(n+1)} &= x_n + \frac{h}{6}(k_{11} + 2k_{21} + 2k_{31} + k_{41}) \\
v_{(n+1)} &= v_n + \frac{h}{6}(k_{12} + 2k_{22} + 2k_{32} + k_{42}) \\
t_{n+1} &= t_n + h
\end{aligned}$$

where, β is representing the parameters like m, k, a, f_0, ω . While, x_n and v_n are the dependent variables, in this case, displacement and velocity, respectively.

Also,

$$f_1(\beta, t_n, x_n, v_n) = v_n$$

&

$$f_2(\beta, t_n, x_n, v_n) = \frac{2.5 \sin(0.5t_n) - 5|v_n|v_n - 6x_n}{2}.$$

Substituting these functions in the above equations and solving gives the required numerical solution for the given second order ODE over the time period of $0 \leq t \leq 15s$.

- (b) The code for the RK-4 method implementation for *stepsize* = 0.5, 0.25, 0.125, 0.0625, 0.03125 is given in cpp file located at folder "q3". We can see from the figures 14 and 16 that decreasing step size from 0.5 to 0.03125 leads to the convergence of displacement and velocity plots to the analytical solution. That means convergence/grid is independent of results.

I have taken differences between the displacements and velocities for consecutive step sizes (*example* : 0.5 and 0.25, ..., 0.0625 and 0.03125) at *time* = 5, 10 and 15secs for which I had given the code in python notebook located at folder "q3". Refer the figures 15 and 17 in which we can clearly see that differences tend to zero as we move towards smaller and smaller step sizes. Thus, convergence is verified.

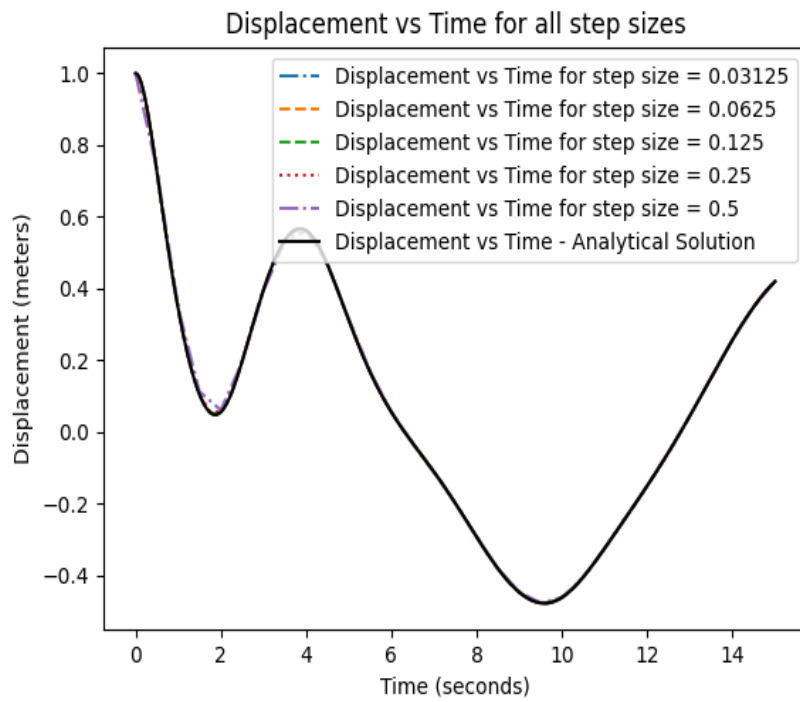


Figure 14: Displacement vs Time for all step sizes

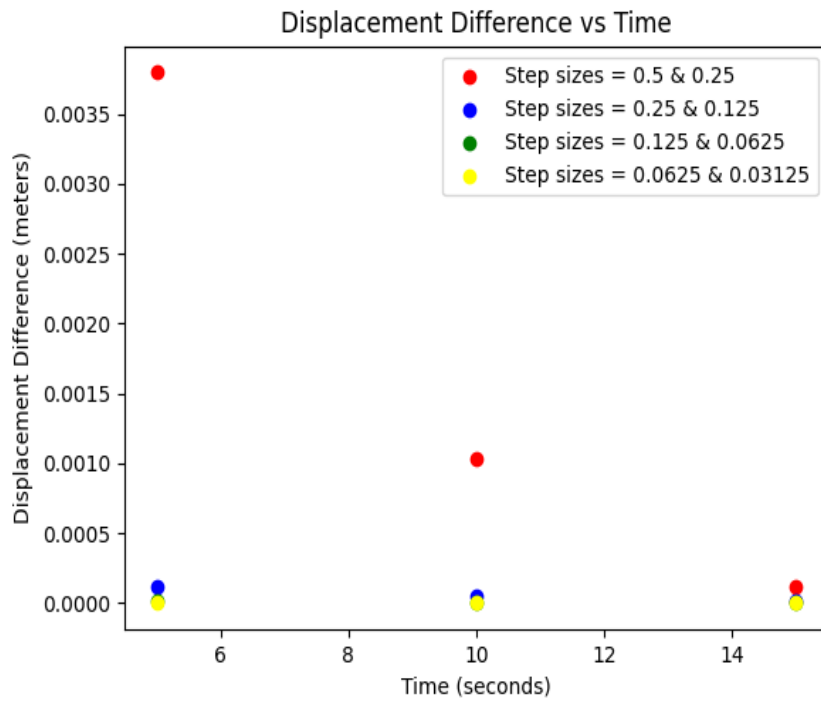


Figure 15: Difference in displacements vs Time for consecutive step sizes

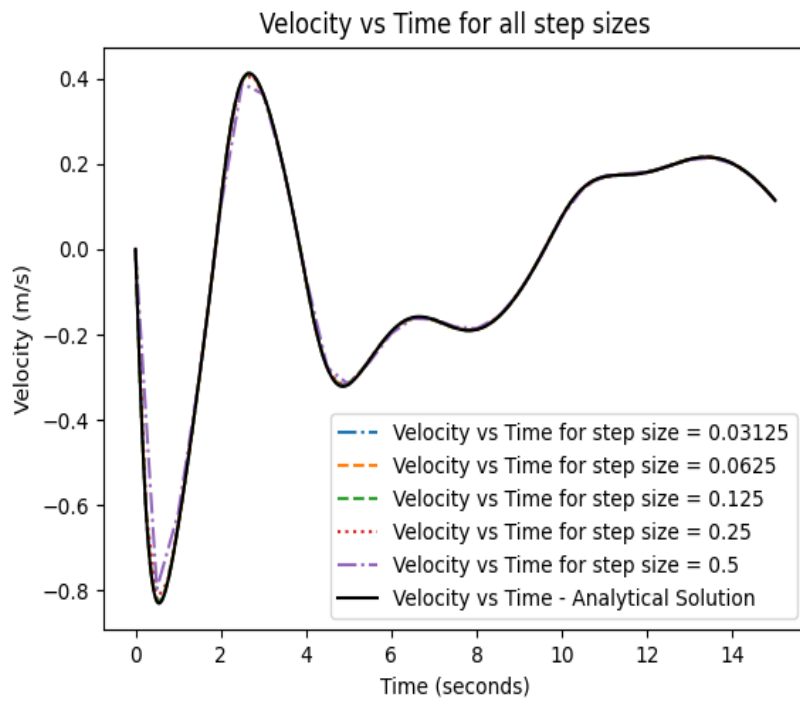


Figure 16: Velocity vs Time for all step sizes

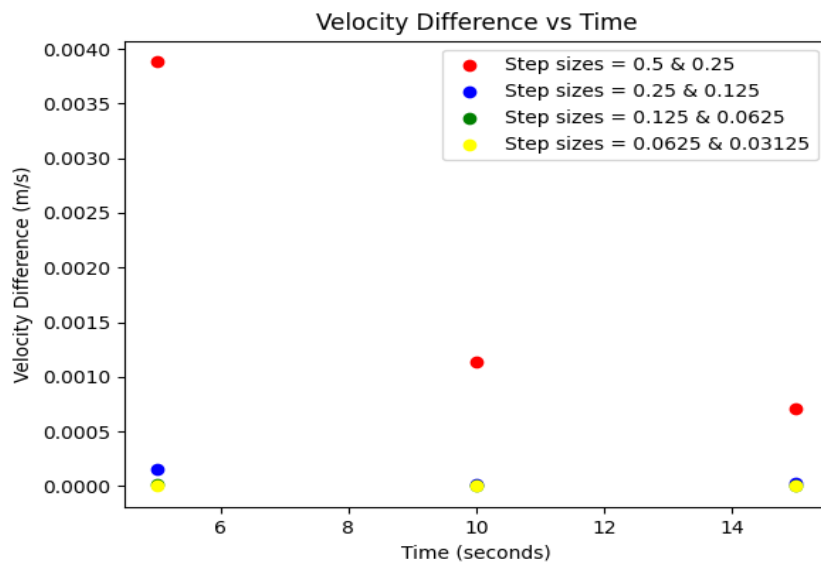


Figure 17: Difference in velocities vs Time for consecutive step sizes

- (c) The maximum and minimum values of velocity and displacement, and corresponding times are obtained using RK-4 method with step size = 0.03125. The code for the same is given in "cpp" file located at folder "q3". Below table 4 is attached for the same:

Quantity	Value	Time (t) (in sec)
Maximum Displacement	1.000000	0.00000
Minimum Displacement	-0.478011	9.59375
Maximum Velocity	0.411387	2.65625
Minimum Velocity	-0.827756	0.56250

Table 4: Maximum and Minimum Values of Displacement and Velocity

Question 4

Given the equation:

$$\frac{dy}{dx} = -1000y + 1000e^{-x} - e^{-x} \quad (2)$$

- (a) Estimate the step-size (Δx_{st}) required to maintain stability using the explicit Euler method.
- (b) With $y(0)$ as 0.5 and 1.0, use the explicit and implicit Euler to obtain a solution from $x = 0$ to 2 using $\Delta x = 0.1$ and Δx_{st} . Compare the solutions from the explicit and implicit Euler schemes.

Solution

- (a) the given ODE is linear. Thus, we can solve it using Leibnitz's method as follows:

$$\frac{dy}{dx} = -1000y + 1000e^{-x} - e^{-x}$$

Rewriting the equation, we get,

$$\frac{dy}{dx} + 1000y = 1000e^{-x} - e^{-x}$$

By comparing with standard form first order LDE,

$$\frac{dy}{dx} + p(x)y = q(x)$$

We get,

$$\text{Integrating factor} = e^{\int p(x) dx} = e^{\int 1000 dx} = e^{1000x}$$

Thus, by multiplying with integrating factor to both sides of equation and then solving the equation, we get,

$$y = e^{-x} + ce^{-1000x}$$

where, c is constant of integration which can be evaluated by the given initial condition.

CASE – I : Initial condition is : $y(0) = 0.5$ by evaluating, we get, $c = -0.5$, hence given LDE solution will be,

$$y = e^{-x} - 0.5e^{-1000x} \quad (3)$$

CASE – II : Initial condition is : $y(0) = 1.0$ by evaluating, we get, $c = 0$, hence given LDE solution will be,

$$y = e^{-x} \quad (4)$$

As we can clearly see that in *case – I* solution to LDE contains the transient term e^{-1000x} , hence it can be classified as *stiff equation*. This stiff equation is decaying, at first solution is dictated by e^{-1000x} , this transient portion of a stiff equation- solution will be rapidly decayed to zero as "x" increases and then after this term decayed significantly, dominance of the solution is governed by e^{-x} . This can be witnessed by the analytical solution plots given in the figures 18 and 19.

The Analytical solution for *case – II* is an exponentially decaying solution. Again, the implicit methods more precisely multi-step implicit methods are preferred to explicit methods for numerically solving the stiff equations. Hence, implicit Euler method is preferred to explicit Euler method, because, implicit euler is unconditionally stable, while explicit euler is stable under some condition. This is proved below.

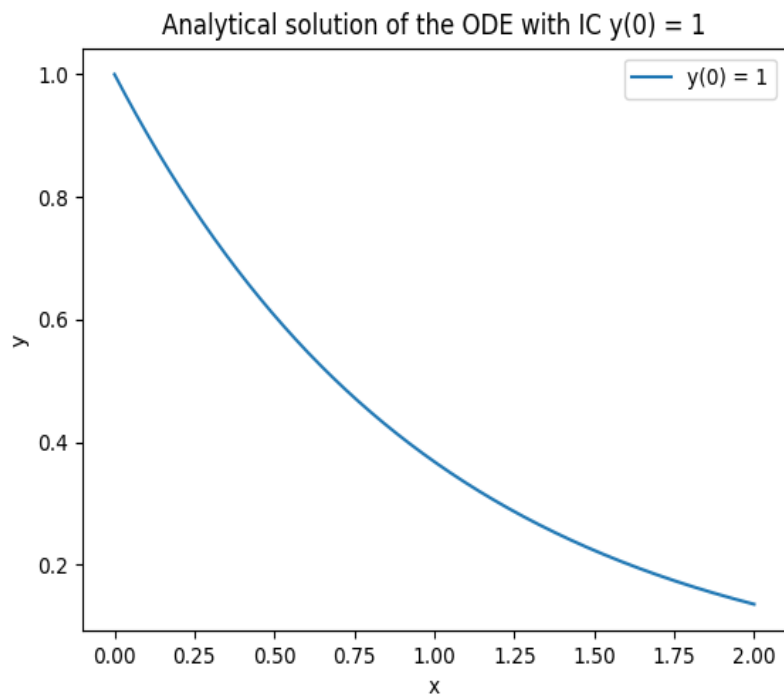


Figure 18: Analytical solution with IC: $y(0) = 1.0$

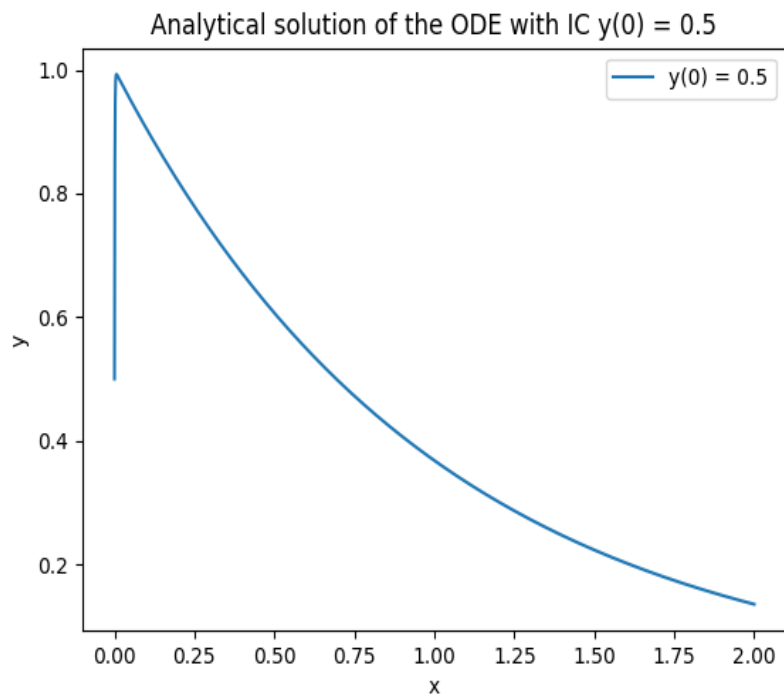


Figure 19: Analytical solution with IC: $y(0) = 0.5$

Explicit Euler method: Explicit Euler method is given by,

$$y_{i+1} = y_i + hf(x_i, y_i)$$

On computer it will be approximated as,

$$w_{i+1} = w_i + hf(x_i, w_i)$$

By subtracting these equations, we get,

$$\epsilon_{i+1} = \epsilon_i + h[f(x_i, y_i) - f(x_i, w_i)]$$

then, factoring out ϵ_i , we get,

$$\frac{\epsilon_{i+1}}{\epsilon_i} = \left[1 + h \frac{\partial f(x_i, y_i)}{\partial y} \right]$$

As, we know errors should be decreasing, hence,

$$\left| \frac{\epsilon_{i+1}}{\epsilon_i} \right| < 1$$

So,

$$\left| 1 + h \frac{\partial f(x_i, y_i)}{\partial y} \right| < 1$$

As, we know,

$$\frac{\partial f(x_i, y_i)}{\partial y} = \frac{\partial[-1000y + 1000e^{-x} + e^{-x}]}{\partial y} = -1000$$

Hence,

$$\begin{aligned} -1 &< 1 - 1000h < 1 \\ \Rightarrow h &< \frac{2}{1000} \end{aligned}$$

Therefore, explicit Euler method is stable when the step size $h < 0.002$.

Implicit Euler method: Implicit Euler method is given by,

$$y_{i+1} = y_i + hf(x_{i+1}, y_{i+1})$$

On computer it will be approximated as,

$$w_{i+1} = w_i + hf(x_{i+1}, w_{i+1})$$

By subtracting these equations, we get,

$$\epsilon_{i+1} = \epsilon_i + h[f(x_{i+1}, y_{i+1}) - f(x_{i+1}, w_{i+1})]$$

then, factoring out ϵ_{i+1} , we get,

$$\frac{\epsilon_{i+1}}{\epsilon_i} = \left[1 + h \frac{\partial f(x_i, y_i)}{\partial y} \right]^{-1}$$

As, we know errors should be decreasing, hence,

$$\left| \frac{1}{1 - h1000} \right| < 1$$

As $h > 0$. Therefore, for any value of h , above condition is satisfied.

Hence, Implicit Euler method is unconditionally stable.

Therefore, the step-size ($\Delta x = h$) required to maintain stability using the explicit Euler method is:

$$\boxed{h_{st} = \Delta x_{st} < 0.002}$$

- (b) The code for implementation of explicit and implicit Euler is given in the cpp file in folder "q4". As we can clearly see from the figures 20 , 23 that error growth is exponential for the step-sizes larger than the (Δx_{st}) .i.e. larger than 0.002

From the figures 21 and 24, we can conclude that initially explicit Euler method make some oscillations and then approximates the analytical solution, in these figures we used step-size equal to (Δx_{st}) .

While from the figures 22 and 25, we can say that numerical solution approximates the true solution very well, ignoring the slight initial oscillations which may arises due to the subtraction of nearly same numbers.

example: As $f(x, y) = -1000y + 1000e^{-x} - e^{-x}$ initially $x = 0.0001$ which leads to the subtraction of $1000e^{-0.0001} - e^{-0.0001} = 1000 \times 0.999900005 - 0.999900005$, here both have the same significant digits, hence this leads to vulnerability to round off errors. Hence, we see the slight oscillations initially.

Finally, we can conclude for the explicit Euler method that:

- (i) $h > 0.002 \Rightarrow$ completely unstable.
- (ii) $0.001 < h < 0.002 \Rightarrow$ partially/oscillatory stable.
- (iii) $h \leq 0.001 \Rightarrow$ completely stable.

As we have discussed that implicit Euler method is unconditionally stable for any step-size. This is witnessed from the figure 26 to figure 31.

In figure 26, segment appears slightly slant at the beginning, whereas in figures 27 and 28, it looks nearly vertical. This happens due to the "*numerical damping*", as I used a gap of 0.1 between the consecutive x-axis points for the figure 26, as a result, it plots relatively with less number of points which could not capture the stiffness well, leading to the slight smoothness, that's why, slightly slant segment appears.

I have used *Newton-Raphson method* as a non-linear solver for the implicit Euler method, code for the same is given in cpp file located at folder "q4".

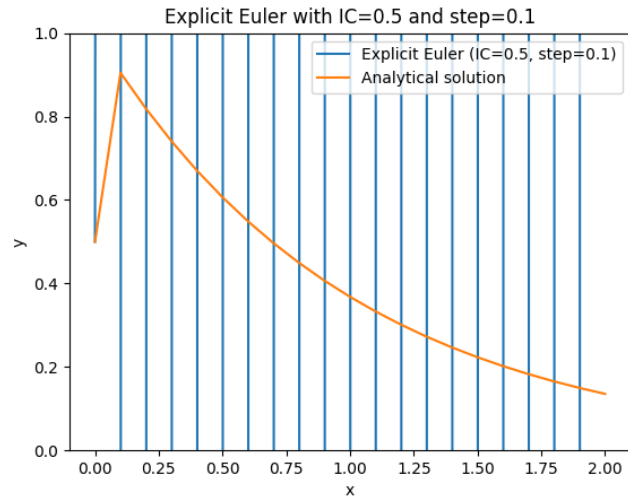


Figure 20: Explicit Euler method with IC: $y(0) = 0.5$, step size $h = 0.1$

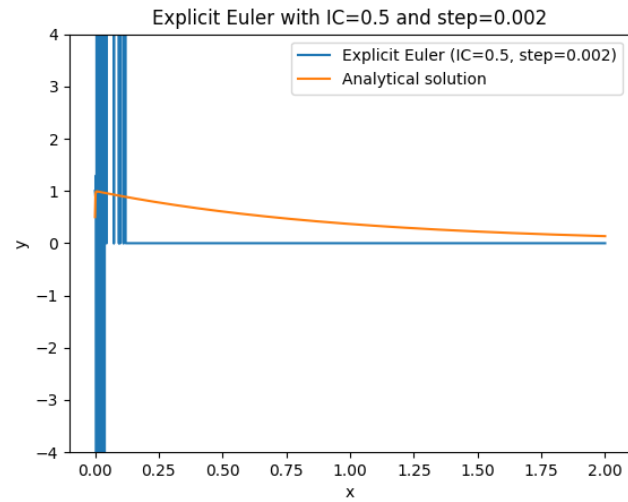


Figure 21: Explicit Euler method with IC: $y(0) = 0.5$, step size $h = 0.002$

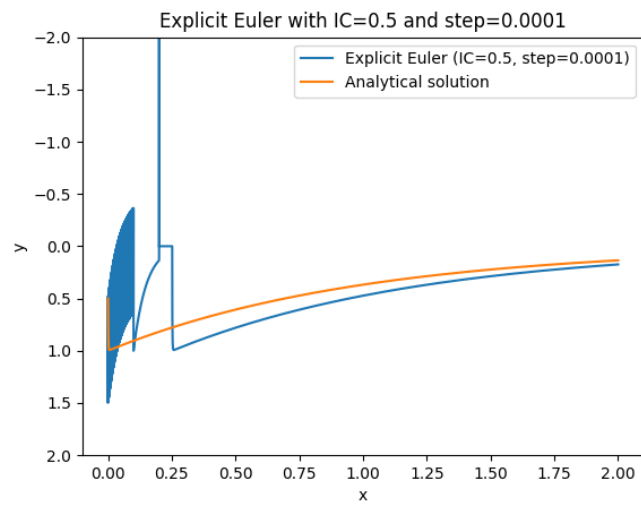


Figure 22: Explicit Euler method with IC: $y(0) = 0.5$, step size $h = 0.0001$

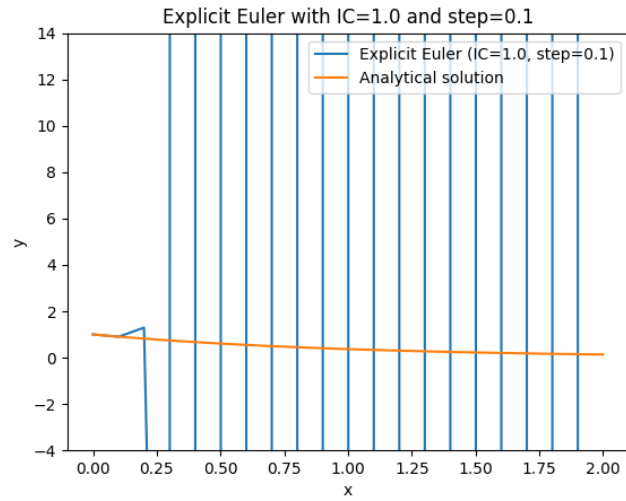


Figure 23: Explicit Euler method with IC: $y(0) = 1.0$, step size $h = 0.1$

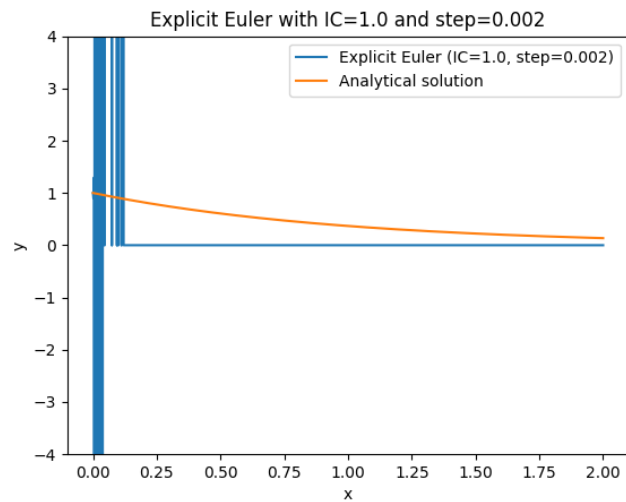


Figure 24: Explicit Euler method with IC: $y(0) = 1.0$, step size $h = 0.002$

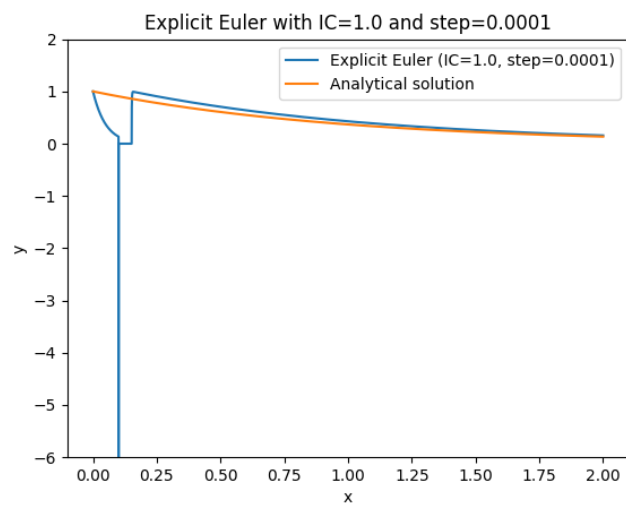


Figure 25: Explicit Euler method with IC: $y(0) = 1.0$, step size $h = 0.0001$

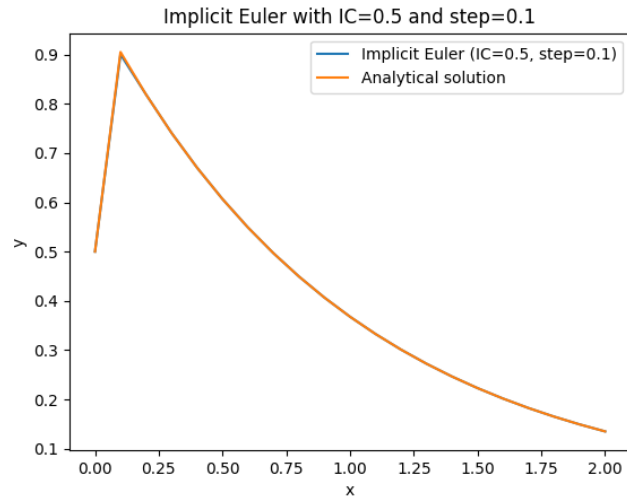


Figure 26: Implicit Euler method with IC: $y(0) = 0.5$, step size $h = 0.1$

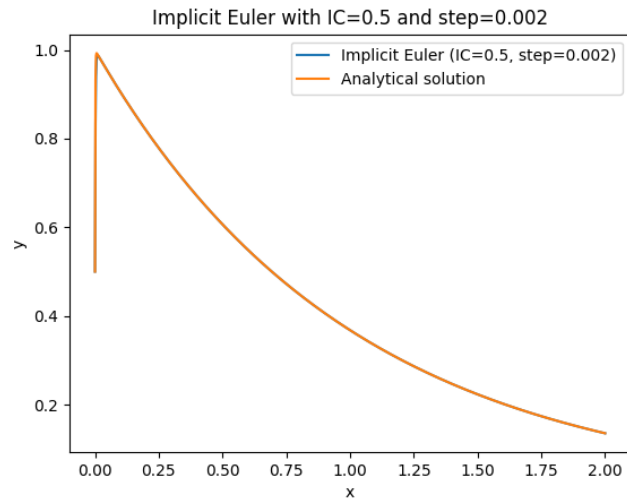


Figure 27: Implicit Euler method with IC: $y(0) = 0.5$, step size $h = 0.002$

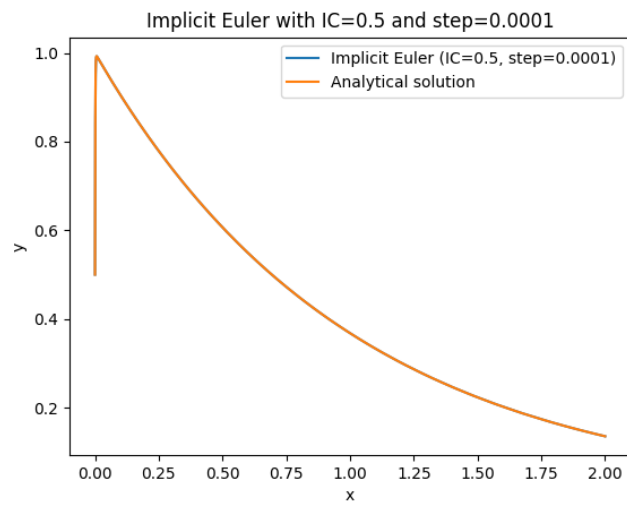


Figure 28: Implicit Euler method with IC: $y(0) = 0.5$, step size $h = 0.0001$

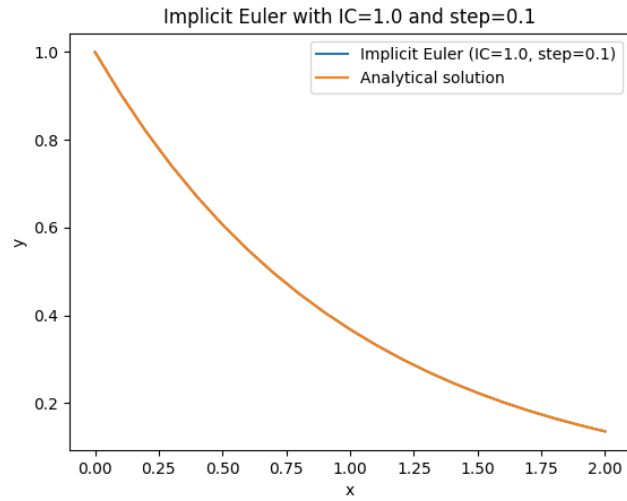


Figure 29: Implicit Euler method with IC: $y(0) = 1.0$, step size $h = 0.1$

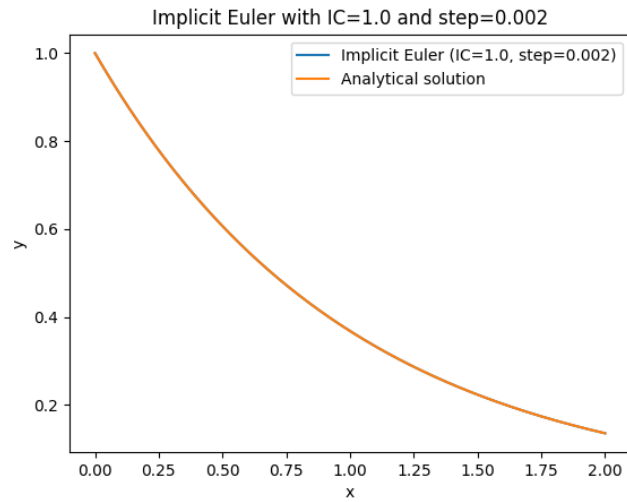


Figure 30: Implicit Euler method with IC: $y(0) = 1.0$, step size $h = 0.002$

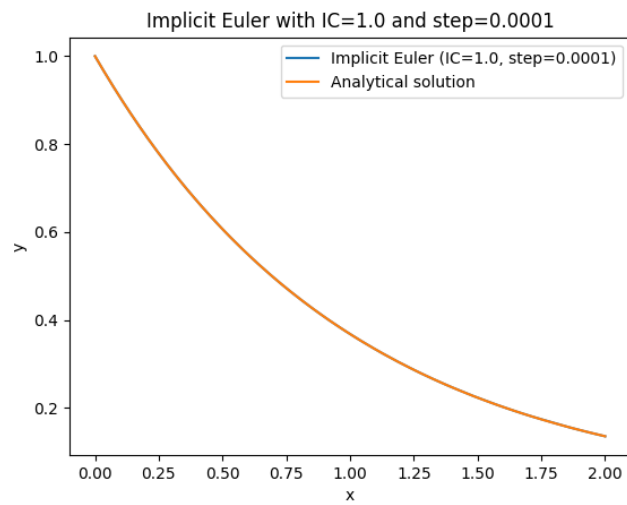


Figure 31: Implicit Euler method with IC: $y(0) = 1.0$, step size $h = 0.0001$

Acknowledgment

I would like to acknowledge the use of ChatGPT and GitHub Copilot for code snippet support in this assignment.